Lab 1: FPGA-based Mental Binary Math Game

Tru Nguyen
PSID: 1911032
ECE 6370

Introduction

This project is the design of a game that can be played by two players on the DE0-CV FPGA board. Each player has one set of four slide switches on the board corresponding to a 4-bit binary number. Player 1 switches are the four rightmost switches (SW3-SW0) which SW0 is the least significant bit. Player 2 switches are the four leftmost switches (SW9-SW6) which SW6 is the least significant bit. Player 1 and Player 2' numbers are displayed on the rightmost 7-segment display and the second to leftmost display in hexadecimal value respectively. The sum of the two players' numbers is displayed on the third 7-segment display from the right in hexadecimal value. The leftmost or rightmost LED is lit if the sum of the two numbers equal 4'b1111 or it doesn't equal respectively.

The rule of the game is that Player 1 sets their number using the switches then Player 2 needs to set a number (one-time only) that the sum of both numbers equals 4'b1111 without looking at Player 1's switches. If Players 2 gets the right number, they win the round. If Player 2 gets the wrong number, Player 1 wins the round. The players swich roles after every round as Player 2 now sets a number for Player 1 to match.

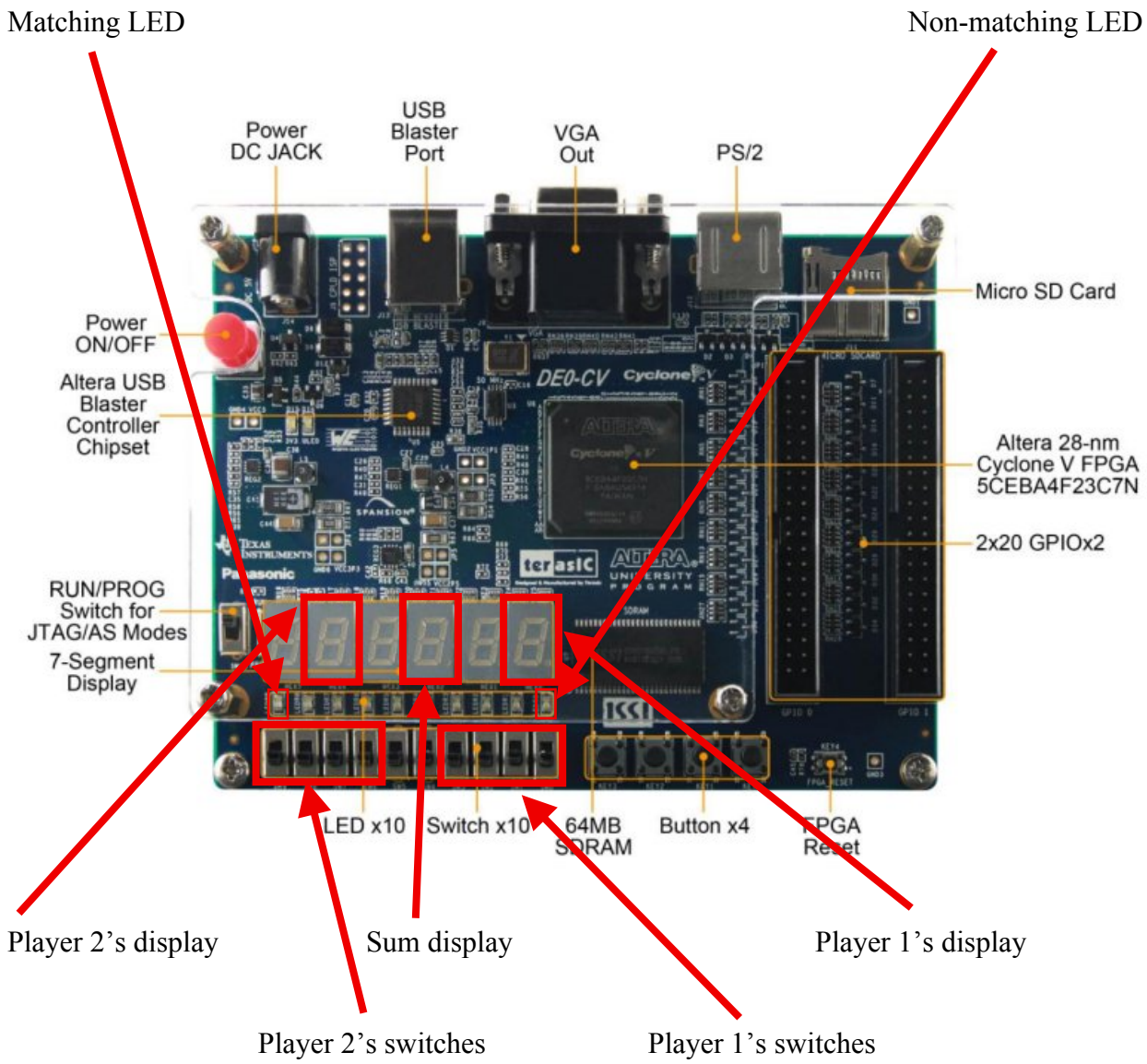

Figure 1: DE0-CV FPGA board with annotated Players' switches, displays, and LEDs

System Architecture Design

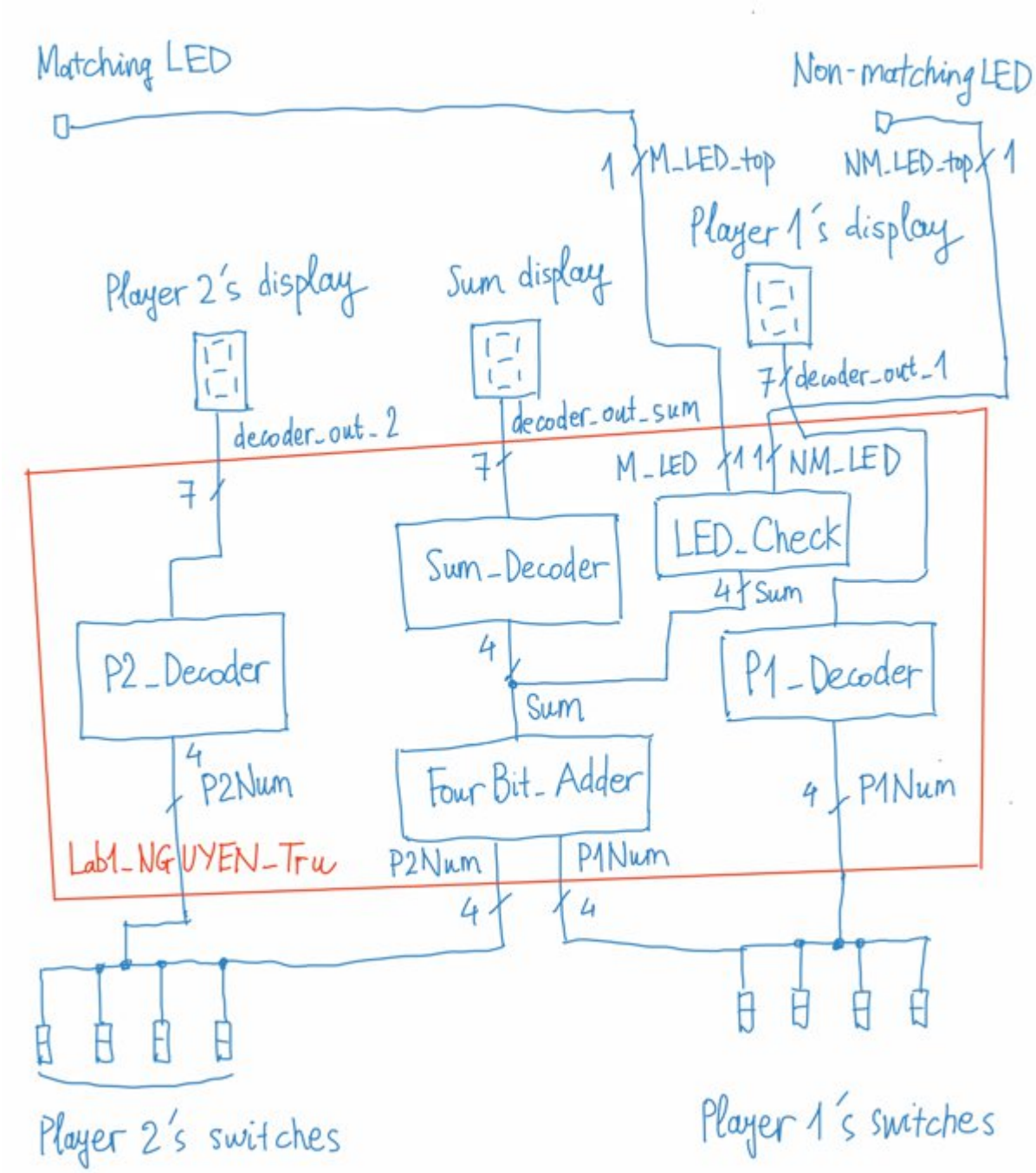

Figure 2: System architecture

The top level system architecture includes five modules: three 7-segment decoder module declared P1_Decoder, P2_Decoder, Sum_Decoder, a 4-bit adder module called FourBit_Adder,

and a verification module called LED_Check. The function of every module and its signal is as follows:

1. P1_Decoder: decodes the 4-bit binary number from Player 1's switches to respective hexadecimal value displayed on Player 1's display.

Input:
[3:0] P1Num: receives Player 1's 4-bit binary number from switches.
Output:
[6:0] decoder_out_1: 7-bit value shows which indexed segment to be turned on and off on Player 1's display.

2. P2_Decoder: decodes the 4-bit binary number from Player 2's switches to respective hexadecimal value displayed on Player 2's display.

Input:
[3:0] P2Num: receives Player 2's 4-bit binary number from switches.
Output:
[6:0] decoder_out_2: 7-bit value shows which indexed segment to be turned on and off on Player 2's display.

3. Sum_Decoder: decodes the 4-bit output binary number from the FourBit_Adder module to respective hexadecimal value displayed on Sum display.

Input:
[3:0] Sum: holds the 4-bit binary sum of Player 1 and 2's 4-bit binary numbers.
Output:
[6:0] decoder_out_sum: 7-bit value shows which indexed segment to be turned on and off on Sum display.

4. FourBit_Adder: performs the sum of Player 1 and 2's 4-bit binary numbers.

Input:
[3:0] P1Num: receives Player 1's 4-bit binary number from switches.
[3:0] P2Num: receives Player 2's 4-bit binary number from switches.
Output:
[3:0] Sum: holds the 4-bit binary sum of Player 1 and 2's 4-bit binary numbers.

5. LED_Check: verifies if the sum of Player 1 and 2's numbers equals 1111 (binary). If so, lights up the leftmost LED or the rightmost LED if otherwise.

Input:
[3:0] Sum: holds the 4-bit binary sum of Player 1 and 2's 4-bit binary numbers.
Output:
M_LED: drives the leftmost (matching) LED based on the sum conditions.
NM_LED: drives the rightmost (non-matching) LED based on the sum conditions.

Simulation Results

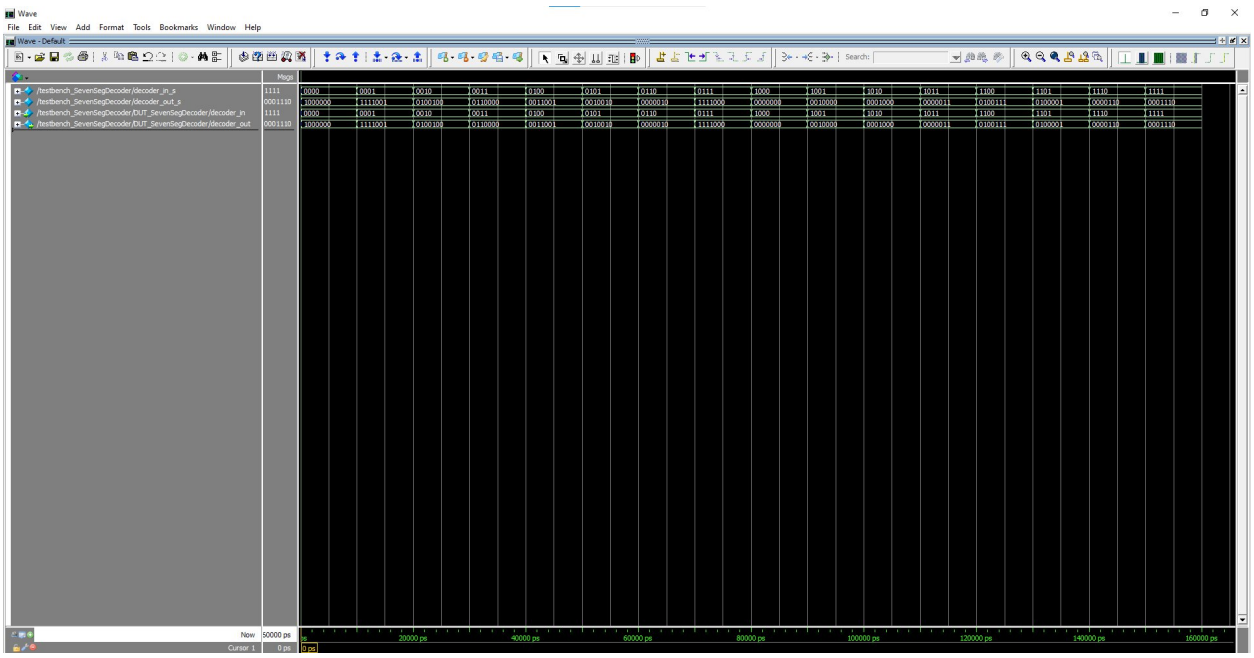

Figure 3: 7-segment decoder simulation results

The 7-segment decoder simulation shows all sixteen cases of a 4-bit binary input number from the FPGA's slide switches and corresponding 7-bit output numbers that display indexed segments on 7-segment displays. This test input number is declared decoder_in_s and test output number decoder_out_s.

Sixteen input test cases are from 0 to F (hex) and corresponding output cases are shown as follows (numbers are in binary):

decoder_in_s = 0000; decoder_out_s = 1000000
decoder_in_s = 0001; decoder_out_s = 1111001
decoder_in_s = 0010; decoder_out_s = 0100100
decoder_in_s = 0011; decoder_out_s = 0110000
decoder_in_s = 0100; decoder_out_s = 0011001
decoder_in_s = 0101; decoder_out_s = 0010010
decoder_in_s = 0110; decoder_out_s = 0000010
decoder_in_s = 0111; decoder_out_s = 1111000
decoder_in_s = 1000; decoder_out_s = 0000000
decoder_in_s = 1001; decoder_out_s = 0010000
decoder_in_s = 1010; decoder_out_s = 0001000
decoder_in_s = 1011; decoder_out_s = 0000011
decoder_in_s = 1100; decoder_out_s = 0100111
decoder_in_s = 1101; decoder_out_s = 0100001
decoder_in_s = 1110; decoder_out_s = 0000110

decoder_in_s = 1111; decoder_out_s = 0001110
default; decoder_out_s = 7'b1000000

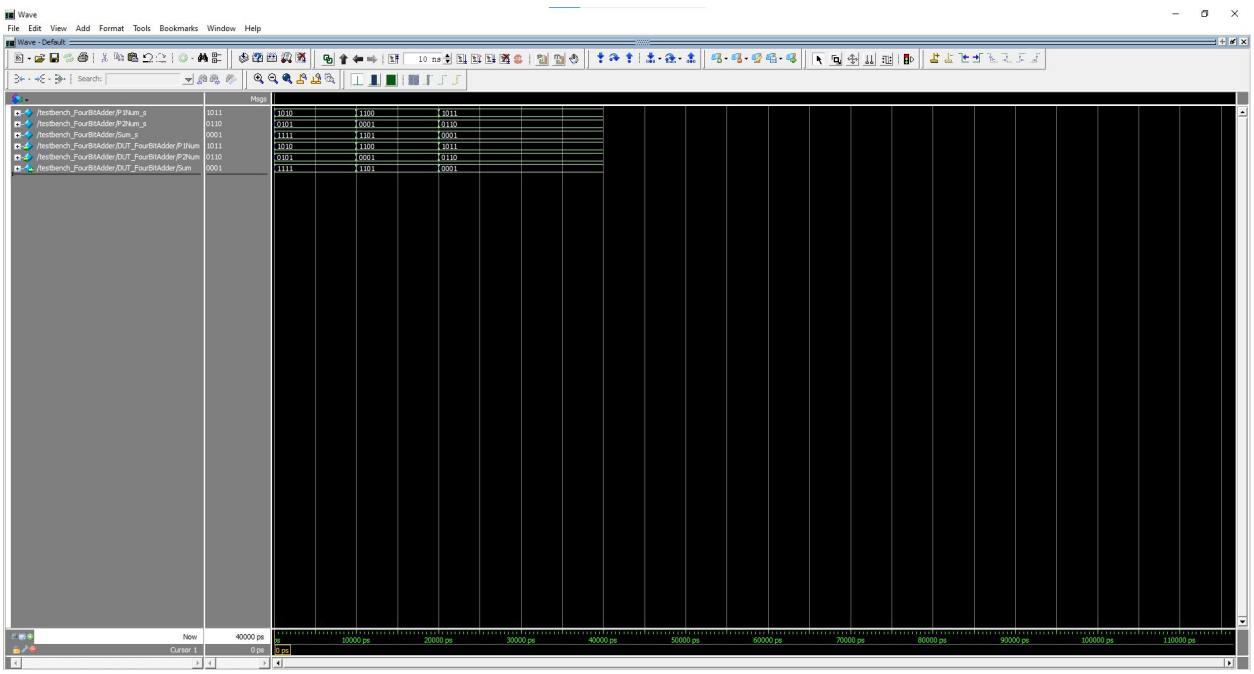

Figure 4: 4-bit adder simulation results

The 4-bit adder simulation shows three sets of test 4-bit input binary numbers from the FPGA's slide switches and corresponding sum of each set. These three sets of test input numbers include a case of matching number, which the sum equals 1111 in binary, and two cases of non-matching numbers. One of the two non-matching cases shows an overflow binary addition, which the output only includes the four least significant bits. These two test input numbers are declared P1Num_s and P2Num_s while the output number is Sum_s.

Three sets of input test cases and corresponding output numbers are as follows (numbers are in binary):

P1Num_s = 1010, P2Num_s = 0101, Sum_s = 1111 (matching case)
P1Num_s = 1100, P2Num_s = 0001, Sum_s = 1101 (non-matching case)
P1Num_s = 1011, P2Num_s = 0110, Sum_s = (1) 0001 (non-matching overflow case)

FPGA Board Testing Results

Step 1: FPGA is powered and in default settings showing both players' numbers are 0000 displayed on respective 7-segment displays. The rightmost LED is lit depicting non-matching condition while the leftmost LED is turned off.

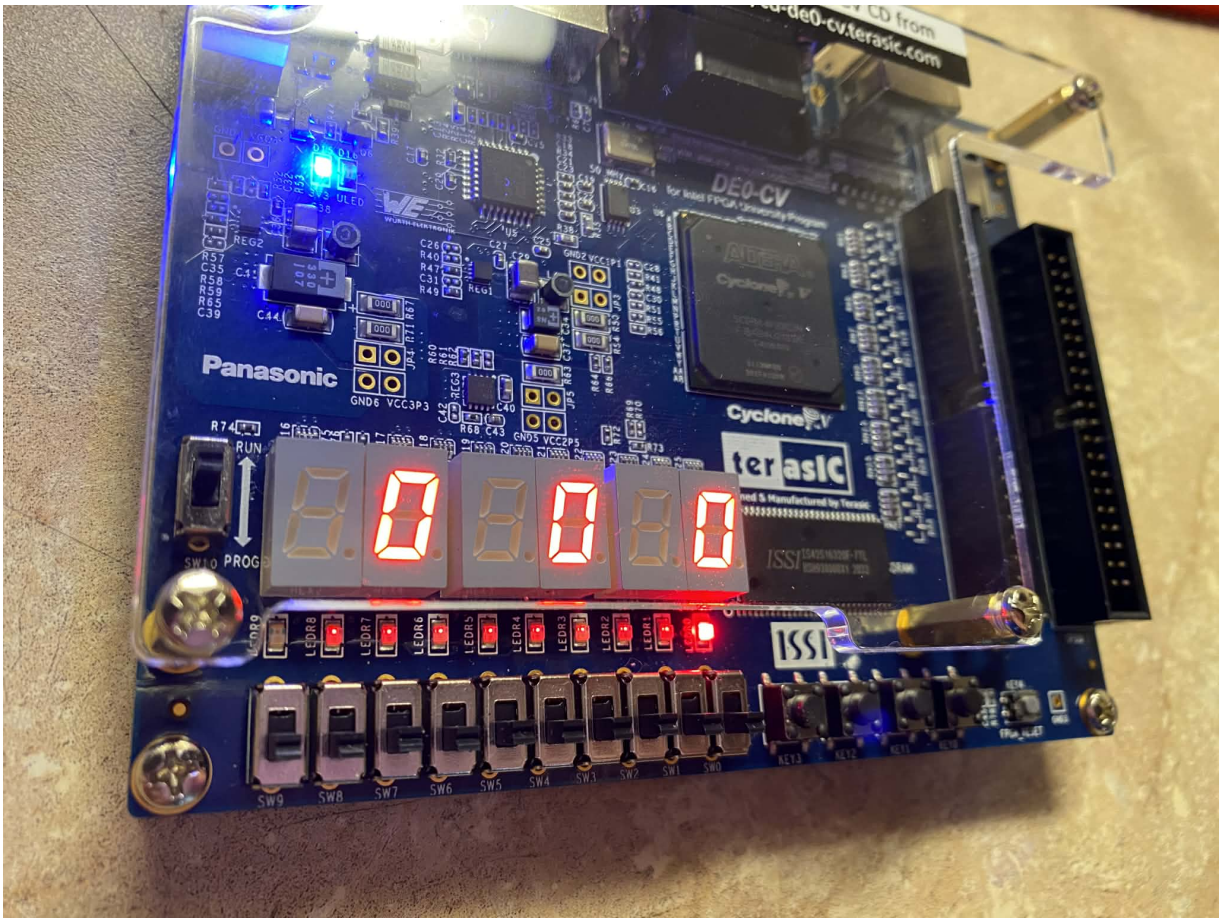

Figure 5: FPGA default settings

Step 2: Player 1 enters their number which is 0110 (6 in hex) and is shown on the right display. The middle display depicts the sum of two numbers which is 0110 in this case because Player 2 hasn't entered their number. The rightmost LED is still lit because of non-matching condition.

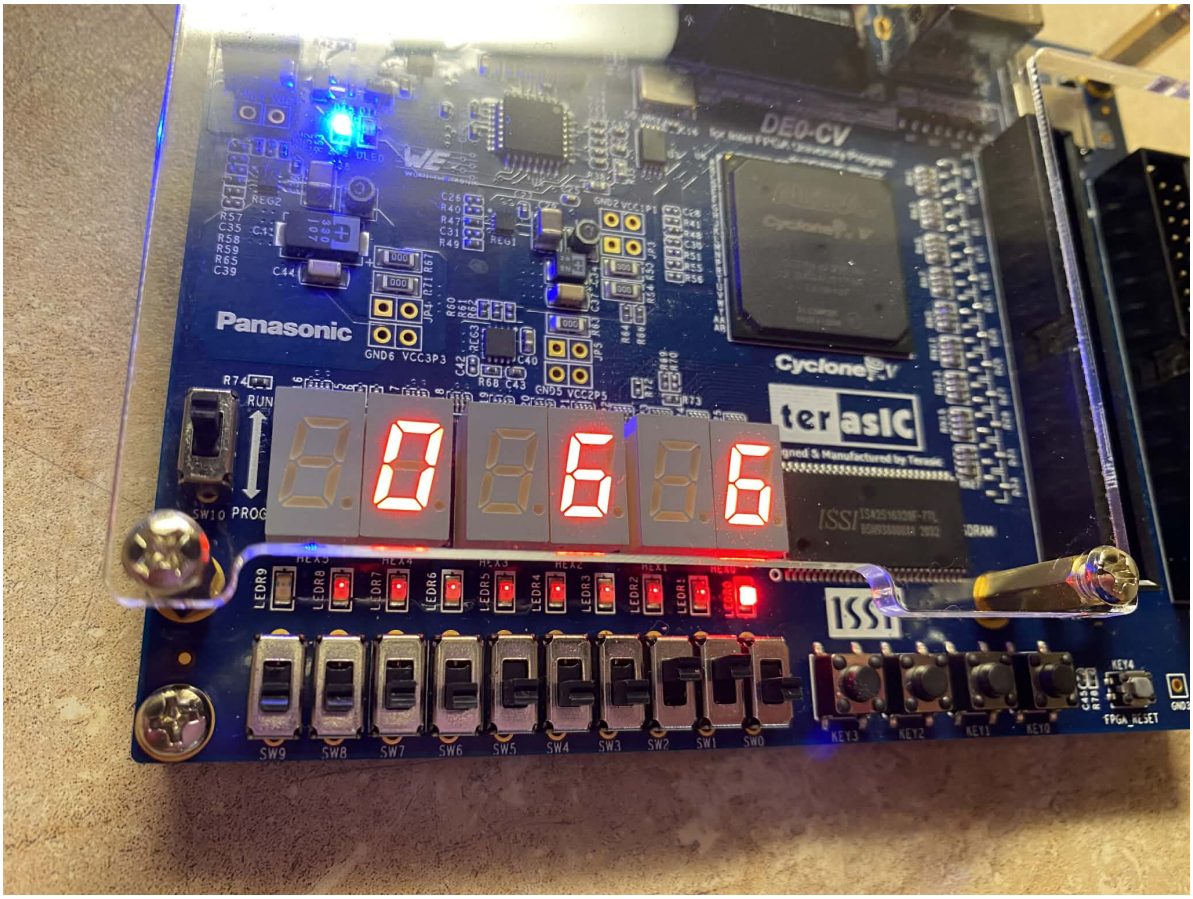

Figure 6: Player 1 number entered

Step 3: Player 2 enters their number which is 1001 (9 in hex) illustrated on the left display. The sum is now 1111 (F in hex). The leftmost LED is now lit up indicating matching condition while the rightmost LED is off. Player 2 is considered the winner of this round.

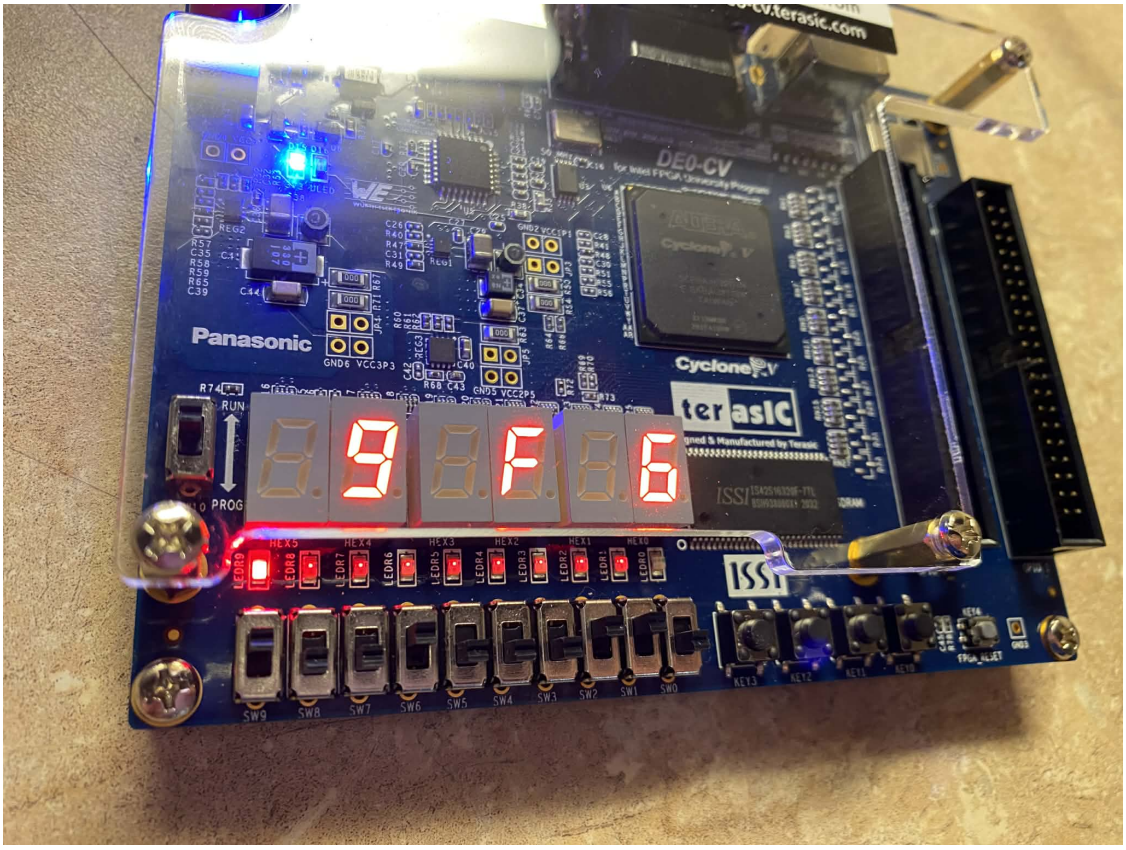

Figure 7: Player 2 number entered and matching condition shown on the FPGA

Step 4: The players switch roles. Player 2 now enters their number first and Player 1 needs to find a matching number. This continues on until the game ends.

Additional: The following figure shows that a non-matching condition occurs because the rightmost LED is lit up. If this happens, the player who entered the matching number loses the round.

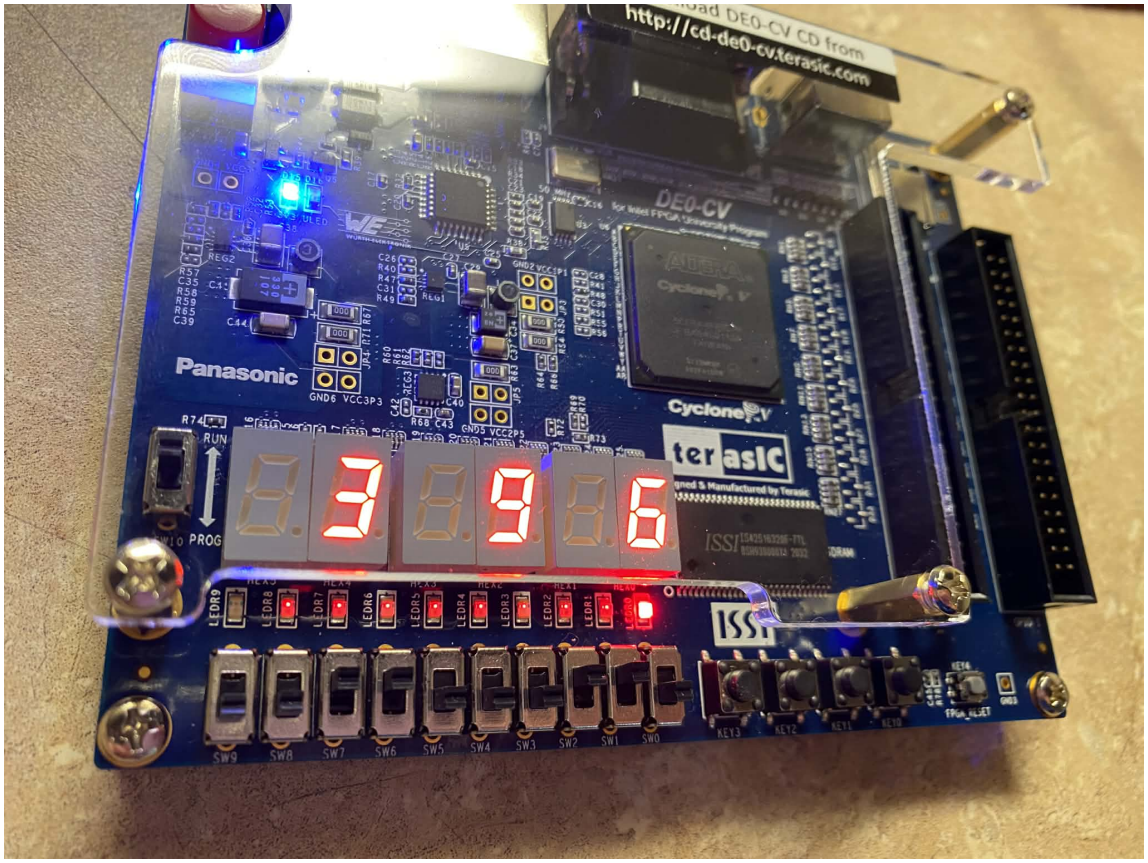

Figure 8: Non-matching condition shown on the FPGA

Video Demo

The demonstration video can be found in the following link:
https://drive.google.com/file/d/1ERW0DEqPbcEXLnO3ceUb6yo0WvSFvyPQ/view?usp=sharing

Conclusion

This project was successfully designed, tested, and implemented on the DE0-CV FPGA board. The bonus features of lighting respective LEDs to indicate matching or non-matching conditions are also addressed and successfully added to the design. The status of this project is complete.

Appendix

```
// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A design of a 7-segment decoder
// This module takes one 4-bit binary input signal decoder_in
and produces a 7-bit output signal decoder_out that
// displays the corresponding input value in hexadecimal on the
FPGA

module SevenSegDecoder(decoder_in, decoder_out);
    input [3:0] decoder_in;
    output [6:0] decoder_out;
    reg [6:0] decoder_out;

    always @(decoder_in)
      begin
        case(decoder_in)
       4'b0000: begin decoder_out = 7'b1000000; end
       4'b0001: begin decoder_out = 7'b1111001; end
       4'b0010: begin decoder_out = 7'b0100100; end
       4'b0011: begin decoder_out = 7'b0110000; end
       4'b0100: begin decoder_out = 7'b0011001; end
       4'b0101: begin decoder_out = 7'b0010010; end
       4'b0110: begin decoder_out = 7'b0000010; end
       4'b0111: begin decoder_out = 7'b1111000; end
       4'b1000: begin decoder_out = 7'b0000000; end
       4'b1001: begin decoder_out = 7'b0010000; end
       4'b1010: begin decoder_out = 7'b0001000; end
       4'b1011: begin decoder_out = 7'b0000011; end
       4'b1100: begin decoder_out = 7'b0100111; end
       4'b1101: begin decoder_out = 7'b0100001; end
       4'b1110: begin decoder_out = 7'b0000110; end
       4'b1111: begin decoder_out = 7'b0001110; end
       default: begin decoder_out = 7'b1111111; end
        endcase
      end

endmodule
```

Figure 9: The 7-segment decoder module

```
`timescale 1 ns/100 ps
module testbench_SevenSegDecoder();

    reg [3:0] decoder_in_s;
    wire [6:0] decoder_out_s;

    SevenSegDecoder DUT_SevenSegDecoder(decoder_in_s,
decoder_out_s);

    initial
      begin
     decoder_in_s = 4'b0000;
     #10 decoder_in_s = 4'b0001;
     #10 decoder_in_s = 4'b0010;
     #10 decoder_in_s = 4'b0011;
     #10 decoder_in_s = 4'b0100;
     #10 decoder_in_s = 4'b0101;
     #10 decoder_in_s = 4'b0110;
     #10 decoder_in_s = 4'b0111;
     #10 decoder_in_s = 4'b1000;
     #10 decoder_in_s = 4'b1001;
     #10 decoder_in_s = 4'b1010;
     #10 decoder_in_s = 4'b1011;
     #10 decoder_in_s = 4'b1100;
     #10 decoder_in_s = 4'b1101;
     #10 decoder_in_s = 4'b1110;
     #10 decoder_in_s = 4'b1111;
      end

endmodule
```

Figure 10: The 7-segment decoder testbench module

```
// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A design of a 4-bit binary adder
// This module takes two 4-bit binary input signals P1Num and
P2Num, calculating the sum of the input signals, and store
// the 4-bit binary result to the output signal Sum

module FourBitAdder(P1Num, P2Num, Sum);
  input [3:0] P1Num, P2Num;
  output [3:0] Sum;
  reg [3:0] Sum;

  always @(P1Num, P2Num)
    begin
      Sum = P1Num + P2Num;

    end
endmodule
```

Figure 11: The 4-bit adder module

```
`timescale 1 ns/100 ps

module testbench_FourBitAdder();

  reg [3:0] P1Num_s, P2Num_s;
  wire [3:0] Sum_s;

  FourBitAdder DUT_FourBitAdder(P1Num_s, P2Num_s, Sum_s);

  initial
    begin
      P1Num_s = 4'b1010; P2Num_s = 4'b0101;  // matching case:
10+5=15
      #10 P1Num_s = 4'hC; P2Num_s = 4'h1; // non-matching case:
12+1=13
      #10 P1Num_s = 4'd11; P2Num_s = 4'd6; // overflow: 11+6=
17 -> (1) 0001


    end
endmodule
```

Figure 12: The 4-bit adder testbench module

```
// ECE 6370
// Author: Tru Nguyen, 1032
// Module: A verification design to check if the sum of two 4-
bit binary numbers equals F
// This module takes one 4-bit binary input signal Sum, checks
if Sum equals F, if so, light up the left-most LED, otherwise
light up the right-most LED.

module LED_OnOff(Sum, M_LED, NM_LED);
  input [3:0] Sum;
  output M_LED, NM_LED;
  reg M_LED, NM_LED;

  always @(Sum, M_LED, NM_LED)
    begin
        if (Sum == 4'b1111)
            begin
              M_LED = 1'b1;
            end
          else
            begin
                NM_LED = 1'b1;
            end
      end
endmodule
```

Figure 13: The verification module of matching and non-matching conditions